

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 4

Roll no: 49 PRN No: 12414235 Name of Student: Saloni Tiwari

Subject Name: Deep Learning

Problem Statement:

Develop an LSTM-based model for time-series forecasting using stock price, weather, or sales datasets.

Algorithm:

- **Import Libraries:** Import TensorFlow/Keras, NumPy, Pandas, Matplotlib, and Scikit-learn for data processing, model creation, training, evaluation, and visualization.
- **Load Dataset:** Load the historical Apple (AAPL) stock price dataset containing daily Date, Open, High, Low, Close, and Volume values.
- **Explore Dataset:** Display the dataset shape, columns, sample records, and visualize the historical closing prices.
- **Preprocess Data:** Select the daily closing price as the target variable, handle missing values, and normalize the prices between 0 and 1 using MinMaxScaler.
- **Create Time-Series Sequences:** Create input sequences using the previous 60 trading days to predict the closing price of the following day.
- **Split the Data:** Divide the sequential data into 80% training and 20% testing sets while maintaining the chronological order of the time-series data.
- **Create LSTM Model:** Build a Sequential LSTM network with two LSTM layers containing 50 units each, Dropout layers for reducing overfitting, and a Dense output layer for predicting the next closing price. Compile it using the Adam optimizer and Mean Squared Error loss.
- **Train the Model:** Train the LSTM model for 20 epochs using a batch size of 32 and validate its performance using the testing data.
- **Analyze Training Performance:** Plot Training Loss and Validation Loss against the number of epochs to analyze the learning behavior and possible overfitting.
- **Perform Prediction:** Use the trained LSTM model to predict closing prices for the test dataset and transform the normalized predictions back to their original price scale.
- **Evaluate the Model:** Calculate Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) to measure the forecasting performance of the LSTM model.

- **Plot Prediction Performance:** Plot Actual vs Predicted stock prices to visually compare the forecasting results.
- **Forecast Next Price:** Use the most recent 60 trading days as input to the trained LSTM model to predict the next closing price.

Code:

```

1] # Importing the required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

2] # Loading and exploring the AAPL stock dataset
url = "https://raw.githubusercontent.com/kristoffer-paulsson/stock-data/master/aapl.csv"
data = pd.read_csv(url, sep=";")
data["date"] = pd.to_datetime(data["date"])
print("Dataset shape:", data.shape)
print("Columns:", data.columns.tolist())
print(data.head())

3] # Visualizing the historical closing price
plt.figure(figsize=(12, 5))
plt.plot(data["date"], data["close"])
plt.title("AAPL Stock Closing Price")
plt.xlabel("Date")
plt.ylabel("Closing Price")
plt.grid()
plt.show()

4] # Preprocessing and normalizing the closing prices
prices = data[["close"]].dropna().values
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_prices = scaler.fit_transform(prices)
print("Number of prices:", len(prices))
print("Normalized data shape:", scaled_prices.shape)

5] # Creating time-series sequences using the previous 60 days
X = []
y = []
for i in range(60, len(scaled_prices)):
    X.append(scaled_prices[i-60:i])
    y.append(scaled_prices[i, 0])
X = np.array(X)
y = np.array(y)
X = X.reshape(X.shape[0], X.shape[1], 1)
print("X shape:", X.shape)
print("y shape:", y.shape)

6] # Splitting the data into training and testing sets
split = int(len(X) * 0.8)
X_train = X[:split]
X_test = X[split:]
y_train = y[:split]
y_test = y[split:]
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

7] # Creating the LSTM neural network
model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(60, 1)),

```

```

        Dropout(0.2),
        LSTM(50),
        Dropout(0.2),
        Dense(1)
    ])
    model.compile(
        optimizer="adam",
        loss="mean_squared_error"
    )
    model.summary()

8] # Training the LSTM model
history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_data=(X_test, y_test),
    verbose=1
)

9] # Plotting training and validation loss
plt.figure(figsize=(10, 5))
plt.plot(
    history.history["loss"],
    label="Training Loss"
)
plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()
plt.grid()
plt.show()

10] # Generating predictions on the test data
predictions = model.predict(X_test)
predictions = scaler.inverse_transform(predictions)
actual = scaler.inverse_transform(
    y_test.reshape(-1, 1)
)
print("Predictions generated successfully")

11] # Evaluating the LSTM model using MAE and RMSE
mae = mean_absolute_error(
    actual,
    predictions
)
rmse = np.sqrt(
    mean_squared_error(
        actual,
        predictions
    )
)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

12] # Plotting actual vs predicted stock prices
plt.figure(figsize=(12, 5))
plt.plot(
    actual,
    label="Actual Price"
)
plt.plot(
    predictions,
    label="Predicted Price"
)

```

```
plt.xlabel("Trading Days")
plt.ylabel("Price")
plt.title("AAPL Stock Price: Actual vs Predicted")
plt.legend()
plt.grid()
plt.show()
```

```
13] # Predicting the next closing price
last_60_days = scaled_prices[-60:]
last_60_days = last_60_days.reshape(
    1, 60, 1
)
next_price = model.predict(
    last_60_days
)
next_price = scaler.inverse_transform(
    next_price
)
print(
    "Predicted next closing price: $",
    round(float(next_price[0][0]), 2)
)
```

Output:

1] Loading and exploring the AAPL stock price dataset.

```
# Load AAPL stock data directly
import pandas as pd

url = "https://raw.githubusercontent.com/kristoffer-paulsson/stock-data/master/aapl.csv"

data = pd.read_csv(url, sep=";")

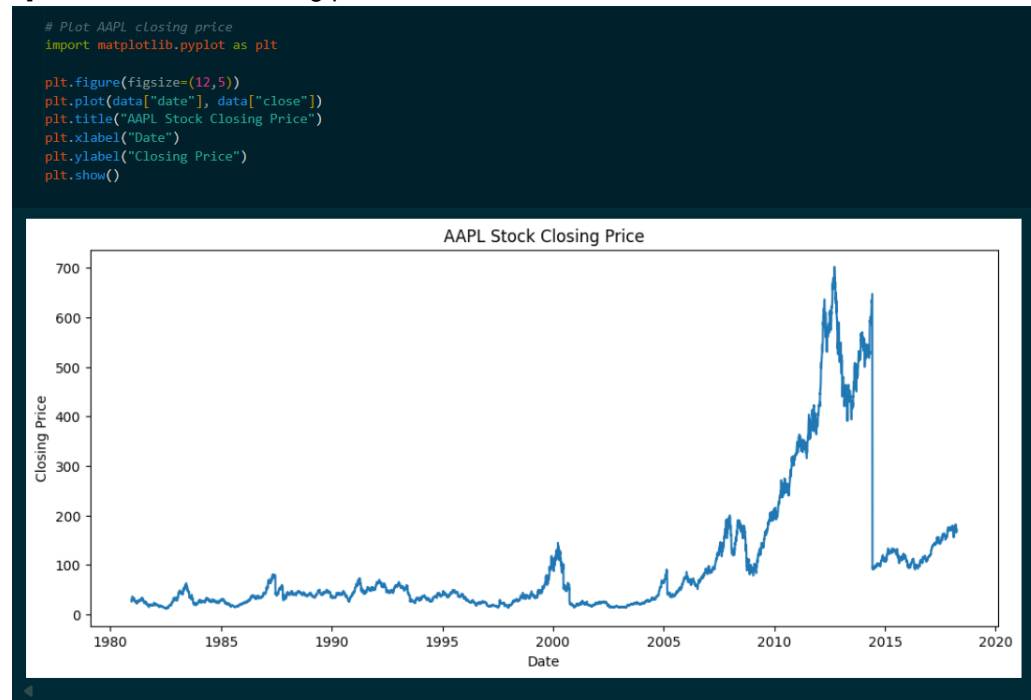
data["date"] = pd.to_datetime(data["date"])

print("Dataset shape:", data.shape)
data.head()
```

Dataset shape: (9400, 6)

	date	open	high	low	close	volume
0	1980-12-12	28.75	28.87	28.75	28.75	2093900.0
1	1980-12-15	27.38	27.38	27.25	27.25	785200.0
2	1980-12-16	25.37	25.37	25.25	25.25	472000.0
3	1980-12-17	25.87	26.00	25.87	25.87	385900.0
4	1980-12-18	26.63	26.75	26.63	26.63	327900.0

2] Historical AAPL closing price visualization.



3] Normalization of closing prices using MinMaxScaler.

```
# Normalize the closing prices
from sklearn.preprocessing import MinMaxScaler

prices = data[["close"]].dropna().values

scaler = MinMaxScaler(feature_range=(0, 1))
scaled_prices = scaler.fit_transform(prices)

print("Number of prices:", len(prices))
```

Number of prices: 9400

```
# Create sequences using the previous 60 days
X = []
y = []

for i in range(60, len(scaled_prices)):
    X.append(scaled_prices[i-60:i, 0])
    y.append(scaled_prices[i, 0])

X = np.array(X)
y = np.array(y)

X = X.reshape(X.shape[0], X.shape[1], 1)

print("X shape:", X.shape)
print("y shape:", y.shape)
```

X shape: (9340, 60, 1)
y shape: (9340,)

```

# Split data into training and testing sets
split = int(len(X) * 0.8)

X_train = X[:split]
X_test = X[split:]

y_train = y[:split]
y_test = y[split:]

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

```

```

Training samples: 7472
Testing samples: 1868

```

4] Creation of 60-day time-series sequences for LSTM input.

```

# Build the LSTM model
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(60, 1)),
    Dropout(0.2),
    LSTM(50),
    Dropout(0.2),
    Dense(1)
])

model.compile(
    optimizer="adam",
    loss="mean_squared_error"
)

model.summary()

```

```

/usr/local/lib/python3.13/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
super().__init__(**kwargs)

```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 50)	10,400
dropout (Dropout)	(None, 60, 50)	0
lstm_1 (LSTM)	(None, 50)	20,200
dropout_1 (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

Total params: 30,651 (119.73 KB)

Trainable params: 30,651 (119.73 KB)

Non-trainable params: 0 (0.00 B)

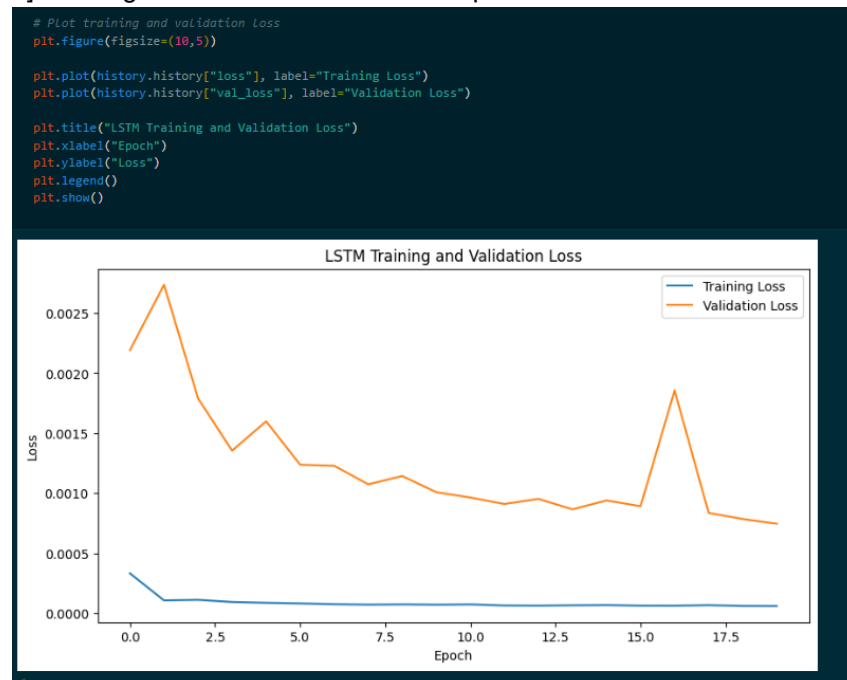
5] Training of the LSTM model for 20 epochs.

```
# Train the LSTM model
history = model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=32,
    validation_data=(X_test, y_test),
    verbose=1
)
```

Epoch 1/20
234/234 ————— 11s 38ms/step - loss: 3.3332e-04 - val_loss: 0.0022
Epoch 2/20
234/234 ————— 7s 32ms/step - loss: 1.0834e-04 - val_loss: 0.0027
Epoch 3/20
234/234 ————— 10s 31ms/step - loss: 1.1313e-04 - val_loss: 0.0018
Epoch 4/20
234/234 ————— 8s 33ms/step - loss: 9.5046e-05 - val_loss: 0.0014
Epoch 5/20
234/234 ————— 10s 34ms/step - loss: 8.8313e-05 - val_loss: 0.0016
Epoch 6/20
234/234 ————— 10s 34ms/step - loss: 8.3171e-05 - val_loss: 0.0012
Epoch 7/20
234/234 ————— 8s 34ms/step - loss: 7.6795e-05 - val_loss: 0.0012
Epoch 8/20
234/234 ————— 10s 32ms/step - loss: 7.3092e-05 - val_loss: 0.0011
Epoch 9/20
234/234 ————— 10s 32ms/step - loss: 7.5197e-05 - val_loss: 0.0011
Epoch 10/20
234/234 ————— 8s 34ms/step - loss: 7.2704e-05 - val_loss: 0.0010
Epoch 11/20
234/234 ————— 10s 34ms/step - loss: 7.4974e-05 - val_loss: 9.6460e-04
Epoch 12/20
234/234 ————— 8s 34ms/step - loss: 6.5929e-05 - val_loss: 9.1071e-04
Epoch 13/20
...
Epoch 19/20
234/234 ————— 8s 33ms/step - loss: 6.2125e-05 - val_loss: 7.8611e-04
Epoch 20/20
234/234 ————— 7s 31ms/step - loss: 6.1321e-05 - val_loss: 7.4643e-04

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

6] Training and validation loss across epochs.



7] Model evaluation using MAE and RMSE metrics.

```
# Generate predictions on the test data
predictions = model.predict(X_test)

predictions = scaler.inverse_transform(predictions)
actual = scaler.inverse_transform(y_test.reshape(-1, 1))

59/59 ————— 1s 12ms/step

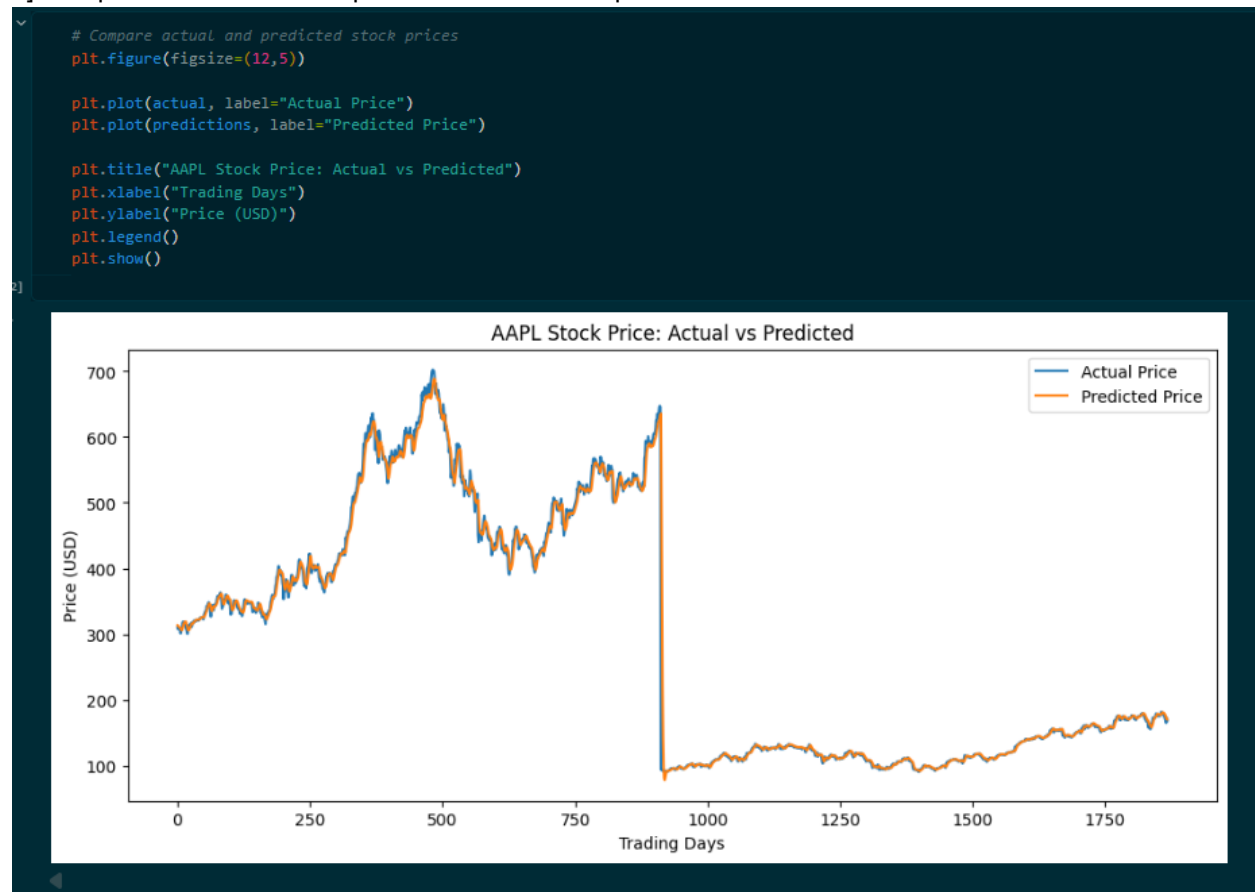
# Evaluate the model using MAE and RMSE
from sklearn.metrics import mean_absolute_error, mean_squared_error

mae = mean_absolute_error(actual, predictions)
rmse = np.sqrt(mean_squared_error(actual, predictions))

print("MAE:", mae)
print("RMSE:", rmse)

MAE: 5.890794357410897
RMSE: 18.881426413283187
```

8] Comparison of actual and predicted AAPL stock prices.



9] Predicted next-day AAPL closing price.

```
# Predict the next closing price
last_60_days = scaled_prices[-60:]
last_60_days = last_60_days.reshape(1, 60, 1)

next_price = model.predict(last_60_days)
next_price = scaler.inverse_transform(next_price)

print("Predicted next closing price: $", round(float(next_price[0][0]), 2))
```

1/1 ————— 0s 32ms/step
Predicted next closing price: \$ 169.85